TECHNICAL EVALUATION · CONFIDENTIALITY CONSCIOUS

NUR FOOD POS

Final Production Readiness & Acquisition Report

Prepared for
Emmanuel Varinot
Managing Director, NEVIE-GLOBAL™

Assessment date: 2 November 2026
Source review and isolated test assessment

NUR FOOD POS

Final Production Readiness & Acquisition Report

Prepared for: Emmanuel Varinot, Managing Director, NEVIE-GLOBAL™

Project: Nur Food POS

Assessment date: 2 November 2026

Repository: `food_software` (workspace snapshot reviewed locally)

Status: Source-level implementation and isolated tests completed; external staging and production operation not verified.

1. Executive Summary

The project has moved beyond a browser-only demonstration in source: a Laravel API now provides persistent, tenant-scoped order, catalog, inventory, shift and report endpoints; the POS frontend calls those endpoints through a same-origin API layer. Automated checks pass: 576 frontend regression assertions, 19 Laravel tests / 101 assertions, API proxy checks, and a Vite production build.

That is not evidence of a production deployment. No live Laravel host, production database, Vercel configuration, staging credentials, physical printer or payment gateway was available. Dependency vulnerability audits could not reach npm or Packagist. No staging deployment was performed. Tenant and branch identifiers are not backed by full company/branch provisioning entities or relational foreign keys. Four-digit PIN authentication and incomplete operations/reporting also remain material risks.

Maturity classification: Prototype / demonstration MVP with an integrated backend foundation (alpha). **Not production-ready or commercially deployable.**

Acquisition readiness: Code can be handed over technically subject to a verified complete repository and lawful ownership; operational transfer readiness is low until staging, provisioning, security, backup/recovery, dependency and legal work is completed.

2. Previous Audit Findings

The prior report classified the product as a prototype/demo MVP. It found the deployed Vercel frontend and Laravel backend were disconnected, business records were browser-local, and there was no demonstrated persistent order workflow. It reported three previously fixed UI/data bugs. This implementation addressed the architectural disconnect in source by adding a Laravel persistence API and same-origin proxy, then changed the POS to use server responses for authentication, catalog, orders, shifts and the primary reports.

The old production URL was not independently verified in this assessment. Source changes do not update the deployed Vercel site. Previous report: `Nur_Food_POS_Technical_Audit_and_Acquisition_Report.md` and its PDF in the repository root.

3. Bugs Corrected

Seven verified problem areas were addressed in this implementation:

1. Browser-local user/PIN state could grant local UI privileges; authentication now relies on the Laravel session endpoint and an HttpOnly session cookie.
2. Orders were treated as browser-local records; checkout now waits for a persistent API response and preserves the cart on backend failure.
3. Frontend-supplied prices and totals could be authoritative; server order creation recalculates from branch catalog prices and tax rates.
4. Orders and catalog operations lacked persistent tenant-scoped API paths; proxy routes and Laravel controllers now implement them.
5. Inventory sale/cancellation lacked a transactional server record; tracked stock now writes movements and valid cancellation restores stock once.
6. Several report panels showed hardcoded sample figures; unsupported panels were changed to empty/limited states, and the primary reports use persisted records.
7. Printer diagnostics could report fictitious connected hardware; UI now states browser-print limitations rather than asserting an unverified device is online.

These are grouped by root cause, not a claim that every defect in every UI path is eliminated. The earlier three fixes remain described in the prior audit and are not double-counted here.

4. Updated Product Maturity

The application has an alpha backend foundation and a legacy static POS interface. It is suitable for continued controlled development and isolated QA. It is not ready to process commercial production sales because real staging, deployment configuration, secure tenant onboarding, persistent operational controls, real payment settlement, full inventory/report workflows and recovery readiness are unverified or absent.

5. Verified POS Features

Capability	Evidence status	Assessment
Independent cart rows, quantities, notes/modifier isolation, VAT-inclusive totals	Automated UI regression checks	Verified in the frontend regression harness; server currently rejects priced modifiers because no authoritative modifier catalog exists.
Orders and order history	Laravel feature tests + frontend harness	Persistent create, server pricing, tenant-scoped history, idempotent retry, cancellation and status update exist. History UI/API is limited to latest 100 orders.
Manual payment recording	Laravel feature tests/source	Server checks declared tender against total. `recorded_manual` is not gateway authorization/settlement. No refund workflow.
Inventory	Laravel feature tests/source	Tracked stock decrement, movement record, adjustment, low-stock response and one-time cancellation restoration exist. Existing catalog defaults do not track stock; full stock operations UI is incomplete.
Main reports	Laravel feature tests/source	Persisted branch/date order totals, product and payment summaries exist. Some UI panels remain local/demo-derived or unsupported.
Authentication/session	Laravel feature tests/source	Four-digit PIN, IP rate limit, Sanctum database token and HttpOnly cookie are implemented. No production identity lifecycle or staging auth test.
Roles/permissions	Laravel feature tests/source	Controller checks and tenant-scoped query filters exist. Role administration and least-privilege production configuration are incomplete.
Shifts	Laravel feature tests/source	Persistent create, scoped listing, active shift and permission/ownership close checks. Closing cash reconciliation is not persisted/verified.
Product/category administration	Laravel tests/source	Branch-scoped catalog reads and protected create/update/category reassignment exist. No persistent image upload or full category hierarchy management workflow.
Customers	Source inspection	Fixed order channels/customer labels only; no customer-management system.
Arabic/English and responsive behavior	Existing frontend regression sources	Cart, layout and language behavior are covered by source-level harnesses; not a deployed cross-device/browser acceptance run.
Payment gateway, physical printers, certified e-invoicing	Not implemented/verified	No gateway/terminal, printer bridge, or independent ZATCA compliance verification.

6. Frontend and Backend Architecture

- **Frontend:** static HTML, CSS and classic JavaScript; Vite 5.4.21 builds distributable assets. Key files include `js/app.js`, `js/payment.js`, `js/store.js`, `js/reports.js`, and `vite.config.js`.
- **Backend:** Laravel/PHP API in `backend/`, controllers under `backend/app/Http/Controllers/Api/`, route declarations in `backend/routes/api.php`.
- **API path:** same-origin `/api/*`; Vercel handlers in `api/` forward allowlisted routes through `api/_proxy.js` to `NURPOS_LARAVEL_API_ORIGIN`. Local Vite development proxies to Laravel.
- **Authentication transport:** Laravel Sanctum token stored in the database; the browser receives an HttpOnly `nurpos_session` cookie; proxy maps it to bearer authorization.
- **Database:** Laravel migrations and Eloquent models. Automated backend tests use isolated in-memory SQLite. A production DB type/instance was not supplied or verified.

The frontend/backend connection is implemented in repository source but **not verified against an actual remote staging deployment**. The Vercel deployment's current configuration and source revision are unknown.

7. Database and Persistent Storage

Orders, line items, shifts, users/tokens, products, categories and inventory movements have database-backed models and migrations. New schema changes include `backend/database/migrations/2026_11_02_000001_create_pos_catalog_and_inventory_tables.php` and `...000002_add_order_lifecycle_fields.php`. Tenant-scoped client order identifiers are unique; relevant query indexes are present.

Limits: no company/branch master tables or provisioning workflow; business tenant references generally use strings without foreign keys. Some references (such as product/category/order/user relationships across tenant keys) rely on controller scoping rather than database composite constraints. No production migration was run. Automated migration behavior was exercised only in tests.

No automated backup, offsite retention, encryption policy, restore drill, RPO/RTO or monitored recovery operation is configured. See `PRODUCTION_READINESS.md` for a safe operational setup checklist. Never run `migrate:fresh` against a business database.

8. Multi-Tenant Security

Order, catalog, inventory, report and shift API queries derive company/branch scope from the authenticated user. Feature tests confirm cross-tenant order/shift reads are not returned and catalog/order operations are branch-scoped. Frontend company/branch values do not determine the API tenant scope.

This is a promising test-backed foundation, not a complete commercial tenancy control plane. There is no company/branch provisioning UI/model, no real deployed cross-tenant penetration test, limited relational tenant constraints, and no production identity lifecycle. Four-digit PIN authentication remains a weak credential model for internet-facing administration even with rate limiting. Production security review and stronger identity controls are required before commercial use.

9. Orders and Payments

Order creation executes within a database transaction, fetches product prices from the user's branch catalog, computes VAT-inclusive tax and discount server-side, records separate line rows, validates tender and can decrement tracked inventory. Client order ID retry is idempotent within company/branch. Cancel checks permission, tenant and order state and restores stock from movement records.

Payment types are manual declarations only. No processor, terminal, settlement confirmation, chargeback, refund or reconciliation integration exists. Product modifiers/addons with price are rejected pending server-side authoritative modifier configuration. Discounts are server-calculated but discount approval policy and cashier limits need definition.

10. Inventory Management

The API supports tracked quantities, adjustments, movement history, low-stock filtering and sale/cancel movement behavior. Database transactions protect stock changes in the tested order lifecycle. However, default catalog seeding does not establish real stock counts, the existing inventory UI has local/demo-derived flows, and no purchasing, supplier receiving, waste approval or branch transfer workflow is verified. Treat stock features as partial.

11. Reporting and Analytics

`backend/app/Http/Controllers/Api/ReportController.php` aggregates actual orders by date window and branch, including order/product/payment summaries. Tests verify persisted branch records are used. Frontend main summaries consume API data.

Several specialized panels (e.g. damage, service durations, commission/application channels, internal orders, balancing and some stock audits) lack authoritative source tables and remain incomplete, unavailable, or local/demo-derived. Shift close does not persist an audited cash count/reconciliation. Report completeness is therefore partial.

12. Authentication and Permissions

Laravel validates PIN format, rate-limits attempts by IP, hashes against stored PBKDF2 hashes, issues Sanctum tokens, and revokes tokens at logout. The browser does not store authoritative user identity in local storage. Session persistence depends on the production database and configured token expiration.

Four-digit PINs provide a small credential space; throttling by IP alone is not a complete control against distributed attempts or shared-network lockouts. No account recovery, forced rotation, MFA, admin provisioning console, audit event pipeline, or operational token revocation UI is provided. Do not expose this API publicly before a security design review.

13. Technology Stack

- JavaScript, HTML and CSS; Node.js/Vite build.
- PHP 8.2 runtime observed locally; Laravel 12 / Sanctum APIs.
- Eloquent and Laravel migrations; local tests use SQLite in memory. Production database is not configured in evidence.
- Vercel static hosting/serverless proxy is configured in source; live project linkage and deployment settings were not inspected.
- npm direct packages: Vite 5.4.21 (MIT), jsdom 30.0.1 (MIT), Puppeteer 25.12.0 (Apache-2.0).
- Composer package inventory includes MIT and BSD-family licensed dependencies. Full notices and transitive obligations require release-time review.

14. External Services and Operating Costs

No actual provider billing account or plan was inspected. An illustrative low-traffic setup might use Vercel Pro (~$20/month), an app VM (~$12/month) and managed 1 GiB database (~$15.15/month), approximately **$47.15/month before taxes, domain, backup/monitoring, overages, support and staging**. These are estimates, not a quote or approved spending. Staging can add a second environment's cost.

References: Vercel pricing, DigitalOcean Droplets pricing, DigitalOcean managed databases pricing. Hardware costs for receipt/kitchen printers or terminals and gateway fees are unknown because no supported integration/provider has been selected. ZATCA/Saudi e-invoicing readiness or certification is not established.

15. Dependency Security Results

`npm audit --omit=dev --audit-level=low` and `composer audit --no-interaction` were attempted. Both could not reach their package registries in this environment. **Vulnerability status is unknown; no clean audit is claimed.** Lockfiles are present, but an online audit and remediation review remain required. Do not upgrade dependencies blindly; review compatible fixes and rerun tests.

Composer license enumeration succeeded locally and shows MIT plus BSD-family licenses. Direct npm package license fields were inspected as listed above. This is not a complete legal license clearance; review all transitive packages, notices, fonts, images and other assets.

16. Test Results

Check	Result	Scope/limit
`node verify_pos.js`	**576 passed, 0 failed**	UI/business-rule regression harness, includes backend rejection preserving cart; mocked API behavior, not deployed browser traffic.
`node verify_api_proxy.js`	**Passed**	Fail-closed config, allowlist, cookie translation, no cookie forwarding and no-store response.
`php artisan test --testdox` (`backend/`)	**19 tests, 101 assertions passed**	Isolated SQLite feature and example tests for authentication, roles, tenant scope, orders, shifts, inventory and reports.
`npm run build`	**Passed**	Vite 5.4.21 production build. Emits warnings about classic scripts not being bundled and deprecated Vite CJS Node API.
npm/Composer security audit	**Unavailable**	Registry network access failed; vulnerability state unknown.
Browser/device/API integration and responsive QA	**Not run against staging**	No deployed staging API/configuration or test credentials provided.
Backup/restore, production DB migration	**Not run**	No production data touched; no staging DB supplied.

Legacy root test scripts include older tests that assume localStorage login or the previous in-memory API. The named current verification commands above are the relevant test evidence for this change; stale scripts should be reconciled before CI adoption.

17. Staging Deployment Status

Not deployed. The code includes environment templates and a Vercel-to-Laravel proxy, but no staging environment, Vercel project configuration, database credentials, API origin, monitoring integration or isolated test dataset was supplied. The proxy intentionally returns 503 when its API origin is unset. Production remains untouched; no code was pushed.

18. Technical Documentation

`PRODUCTION_READINESS.md` documents architecture, setup, API paths, data/access design, backup/recovery requirements, limitations, verification evidence, cost assumptions and the production gate. `README.md` and `backend/README.md` also describe local setup and limitations. API documentation is route/source based; no generated OpenAPI specification exists.

19. Remaining Issues and Limitations

1. No staging deployment or live end-to-end frontend/API/database verification.
2. No production database, user/company/branch provisioning, backup automation or restore drill.

3. Four-digit PIN auth is not sufficient as a finished internet-facing identity system; no MFA/account lifecycle.
4. Tenant isolation lacks complete relational company/branch entities and database constraints; deployed penetration testing is outstanding.
5. Payment gateway, terminal, refunds and settlement are absent; payment records are manual.
6. Modifier pricing is unsupported by server catalog and rejected during order submission.
7. Inventory/reporting/shift reconciliation workflows are only partially backed by authoritative data.
8. Specialized reports and customer management are incomplete; some legacy local/demo panels remain.
9. No real receipt/kitchen printer bridge or persistent product image storage.
10. npm/Composer vulnerability audits blocked by registry connectivity; audit status unknown.
11. ZATCA/Saudi e-invoicing compliance and certification are unverified.
12. No complete contributor/IP chain-of-title or asset license review; repository access does not prove ownership or exclusivity.
13. Legacy test scripts and some unused/local-only modules need cleanup and CI classification.

20. Acquisition Readiness Assessment

Technical source handover: conditionally feasible after repository completeness, environment templates, third-party notices, contributor history and legal rights are confirmed. No credentials are embedded in this report. The checkpoint branch was created locally before implementation; changes have not been pushed.

Independent commercial operation: not ready. A buyer would need to provision and secure Laravel hosting/database, create tenant/admin provisioning, configure HTTPS origin and secrets, complete staging integration, choose backup/recovery and monitoring, perform online dependency/security review, add supported payment/printing/regulatory integrations as required, and run operational acceptance tests.

No acquisition price, monthly development arrangement or transaction is approved by this report. Exclusive ownership and transfer rights require legal confirmation from all relevant contributors and third-party licensors.

Final Technical Conclusion

The work materially improves the source architecture and verifies core order, session, catalog, stock and report API paths under automated isolated tests. It does not demonstrate production operation. Nur Food POS should remain classified as an **alpha prototype / demonstration MVP**, with **low operational acquisition readiness** pending the blockers above. Commercial deployment should wait for a verified isolated staging release, stronger production identity controls, full tenant model review, dependency audit, backup/restore drills, payment and hardware decisions, and independent security/legal review.